

1) Priority scheduling non-pre-emptive

```
#include<stdio.h>
int main()
{
    int n,i,time=0,completed=0,idx;

    int pid[20],at[20],bt[20],pr[20];
    int ct[20],wt[20],tat[20],done[20];

    float avg_wt=0,avg_tat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        pid[i]=i+1;

        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);

        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&pr[i]);

        done[i]=0;
    }

    while(completed<n)
    {
        idx=-1;

        for(i=0;i<n;i++)
        {
            if(at[i]<=time && done[i]==0)
            {
                if(idx==-1 || pr[i] < pr[idx])
                {
                    idx=i;
                }
            }
        }
    }
```

```

    }

    if(idx!=-1)
    {
        time += bt[idx];
        ct[idx] = time;

        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        avg_wt += wt[idx];
        avg_tat += tat[idx];

        done[idx]=1;
        completed++;
    }
    else
    {
        time++;
    }
}

printf("\nProcess\tAT\tBT\tPR\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],wt[i],tat[i]);
}

avg_wt = avg_wt/n;
avg_tat = avg_tat/n;

printf("\nAverage Waiting Time = %.2f",avg_wt);
printf("\nAverage Turnaround Time = %.2f\n",avg_tat);

return 0;
}

```

```
C:\Users\Devi\Desktop\neha >
Enter number of processes: 4
Enter Arrival Time for P1: 1
Enter Burst Time for P1: 1
Enter Priority for P1: 2
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 3
Enter Priority for P2: 4
Enter Arrival Time for P3: 1
Enter Burst Time for P3: 0
Enter Priority for P3: 7
Enter Arrival Time for P4: 0
Enter Burst Time for P4: 3
Enter Priority for P4: 5

Process AT    BT    PR    WT    TAT
P1      1      1      2      2      3
P2      2      3      4      2      5
P3      1      0      7      6      6
P4      0      3      5      0      3

Average Waiting Time = 2.50
Average Turnaround Time = 4.25

Process returned 0 (0x0)   execution time : 31.604 s
Press any key to continue.
```

2) Priority scheduling pre-emptive

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, time = 0, completed = 0, choice;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n], pr[n];
```

```
    int rt[n], ct[n], wt[n], tat[n];
```

```
    printf("Enter priority type (1: Higher number = Higher priority, 2: Lower number = Higher priority): ");
```

```
    scanf("%d", &choice);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("\nEnter Process ID: ");
```

```
        scanf("%d", &pid[i]);
```

```
printf("Enter Arrival Time: ");
scanf("%d", &at[i]);
```

```
printf("Enter Burst Time: ");
scanf("%d", &bt[i]);
```

```
printf("Enter Priority: ");
scanf("%d", &pr[i]);
```

```
    rt[i] = bt[i];
}
```

```
while(completed < n)
```

```
{
```

```
    int selected = -1;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(at[i] <= time && rt[i] > 0)
```

```
        {
```

```
            if(selected == -1)
```

```
                selected = i;
```

```
            else
```

```
            {
```

```
                if(choice == 1) // Higher number = higher priority
```

```
                {
```

```
                    if(pr[i] > pr[selected])
```

```
                        selected = i;
```

```
                }
```

```
                else // Lower number = higher priority
```

```
                {
```

```
                    if(pr[i] < pr[selected])
```

```
                        selected = i;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    if(selected == -1)
```

```
    {
```

```
        time++;
```

```
    }
```

```
    else
```

```
    {
```

```

        rt[selected]--;
        time++;

        if(rt[selected] == 0)
        {
            completed++;
            ct[selected] = time;
            tat[selected] = ct[selected] - at[selected];
            wt[selected] = tat[selected] - bt[selected];
        }
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
printf("Average Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

```
C:\Users\Def\Desktop\Untitled x + v
Enter number of processes: 3
Enter priority type (1: Higher number = Higher priority, 2: Lower number = Higher priority):
1
Enter Process ID: 1
Enter Arrival Time: 2
Enter Burst Time: 3
Enter Priority: 5

Enter Process ID: 0
Enter Arrival Time: 1
Enter Burst Time: 2
Enter Priority: 3

Enter Process ID: 2
Enter Arrival Time: 4
Enter Burst Time: 6
Enter Priority: 9

PID    AT    BT    PR    CT    TAT    WT
P1      2     3     5     11     9     6
P0      1     2     3     12    11     9
P2      4     6     9     10     6     0

Average Turnaround Time = 8.67
Average Waiting Time = 5.00

Process returned 0 (0x0)   execution time : 31.027 s
Press any key to continue.
```

3) Round Robin

```
#include <stdio.h>
```

```
int main()
{
    int n, tq;
    int at[20], bt[20], rt[20];
    int ct[20], wt[20], tat[20];

    int i, time = 0, done, completed = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);

        rt[i] = bt[i];
    }
}
```

```

printf("\nEnter Time Quantum: ");
scanf("%d",&tq);

while(completed < n)
{
    done = 1;

    for(i=0;i<n;i++)
    {
        if(at[i] <= time && rt[i] > 0)
        {
            done = 0;

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                ct[i] = time;
                rt[i] = 0;
                completed++;

                tat[i] = ct[i] - at[i];
                wt[i] = tat[i] - bt[i];
            }
        }
    }

    if(done)
        time++;
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
}

```

```

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nAverage Turnaround Time = %.2f",avg_tat);
    printf("\nAverage Waiting Time = %.2f",avg_wt);

    return 0;
}

```

```

C:\Users\Devi\Desktop\Untitled - [icon] + - x
Enter number of processes: 3
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 2
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 4
Enter Burst Time for P3: 6
Enter Time Quantum: 2
PID   AT   BT   CT   TAT  WT
P1     0   2    2    2    0
P2     2   3    7    5    2
P3     4   6   11    7    1
Average Turnaround Time = 4.67
Average Waiting Time = 1.00
Process returned 0 (0x0)   execution time : 12.020 s
Press any key to continue.

```